

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №3

Выполнил:
Губанов Егор
Группа: БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026

Задача

реализовать тестирование API средствами Postman;
написать тесты внутри Postman.

В результате хочу получить один комплексный сценарий по основному процессу в вашем приложении.

Пример такого сценарий для системы по аренде недвижимости: регистрация -> вход -> список объявлений -> фильтрация по списку -> просмотр случайного объявления из списка -> старт переписки по объявлению.

Необходимо сделать отчёт по шаблону

Ход работы

1. Объект тестирования и инструменты

Параметр	Значение
Сервис	REST API сервиса аренды недвижимости (Lab1)
Базовый URL	{{baseUrl}} → http://localhost:3000
Префикс API	/api/v1
Инструмент	Postman (коллекция + Collection Runner)
Файл коллекции	homeworks/hw3/Rent_Lab1_E2E.postman_collection.json
Спецификация	OpenAPI из Д32 (для сверки путей и тел); Swagger UI на localhost:8001/docs

2. Структура коллекции

Коллекция организована в три папки, рассчитанных на последовательный прогон:

- «Д33 - один проход» - сквозной сценарий для отчёта и демо; запускается одним прогоном через Collection Runner. Покрывает регистрацию, создание объекта, листинг, фильтрацию, переписку и прочтение сообщений.
- «Owner» - короткие проверки после основного сценария: профиль, обновление токена через refresh, история арендатора.
- «Остальной API» - расширенное покрытие оставшихся эндпоинтов (фото, условия аренды, сделки, DELETE, logout, негативные кейсы). Зависит от переменных, заполненных первой папкой.

3. Переменные коллекции

Все запросы используют переменные коллекции вместо жёстко заданных значений. Это позволяет менять хост и многократно прогонять сценарий без ручного редактирования URL и заголовков.

Переменная	Назначение
baseUrl	Хост API. Смена порта/машины без правки каждого URL.
ownerEmail / tenantEmail	Уникальные email (генерируются в Pre-request Script через timestamp).
ownerToken / tenantToken	Bearer-токены; заполняются в Tests после регистрации/входа.
ownerRefreshToken	Refresh-токен владельца для сценария обновления сессии.
ownerId / tenantId	UUID пользователей; используются для проверок и фильтрации.
typeId	ID первого типа недвижимости из GET /property-types.
propertyId	ID созданного объекта (POST /properties).
scenarioCity	Город объекта; используется в фильтрующем GET /properties.

pickedPropertyId / pickedOwnerId	ID объекта и владельца, выбранных после листинга для переписки.
messageId	ID первого сообщения арендатора для PATCH .../messages/:id/read.
dealId / dealStart / dealEnd	Параметры сделки; даты генерируются в Pre-request Script.
photoId / conditionId	ID фото и условия аренды для DELETE-запросов.

Email-адреса генерируются уникально через Pre-request Script с использованием Date.now(), что исключает конфликты при повторных прогонах.

4. Сквозной сценарий (папка «ДЗЗ - один проход»)

Папка содержит 14 запросов, которые выполняются строго по порядку. Каждый шаг передаёт данные следующему через переменные коллекции.

№	Запрос	Действие	Tests / переменные
1	GET /property-types	Получить справочник типов недвижимости	Сохранить typeId первого типа
2	POST /auth/register (owner)	Зарегистрировать пользователя с ролью OWNER	Сохранить ownerToken, ownerRefreshToken, ownerId
3	POST /auth/register (tenant)	Зарегистрировать пользователя с ролью TENANT	Сохранить tenantToken, tenantId
4	POST /properties	Создать опубликованный объект (СПб, цена в диапазоне фильтра)	Сохранить propertyId, scenarioCity
5	GET /properties (без токена)	Публичный листинг с пагинацией	Проверка структуры: items[], total
6	GET /properties (фильтр)	Фильтр: город + тип + диапазон цены	Сохранить pickedPropertyId, pickedOwnerId
7	GET /properties/:id	Публичная карточка объекта	Проверка полей: title, city, price, photos
8-12	POST /messages (x5)	Переписка арендатор ↔ владелец (чередование)	Сохранить messageId первого сообщения
13	GET /messages?property_id=...	Листинг сообщений по объекту (order=asc)	Проверка: не менее 6 сообщений, наличие маркеров в тексте
14	PATCH /messages/:id/read	Владелец помечает прочитанным первое сообщение	Статус 200, is_read: true

5. Автопроверки (вкладка Tests)

Каждый запрос содержит автопроверки во вкладке Tests. Типовые паттерны:

Проверка статуса ответа:

```
pm.test("Status 200", () => {
  pm.response.to.have.status(200);
});
```

Разбор тела и сохранение переменных:

```
const json = pm.response.json();
pm.collectionVariables.set("ownerToken", json.access_token);
pm.collectionVariables.set("ownerId", json.user.id);
```

Проверка структуры ответа:

```
pm.test("items is array", () => {  
  pm.expect(json.items).to.be.an("array");  
  pm.expect(json.total).to.be.a("number");  
});
```

Проверка содержимого треда сообщений:

```
pm.test("Thread has 6+ messages", () => {  
  pm.expect(json.items.length).to.be.at.least(6);  
});
```

Негативный кейс (ожидается 401):

```
pm.test("No token → 401", () => {  
  pm.response.to.have.status(401);  
});
```

6. Папка «Owner»

Запросы выполняются после основного сценария и проверяют корректность токенов и сессии:

Запрос	Цель
GET /me (owner)	Проверка профиля владельца по Beager-токену.
POST /auth/login (owner)	Перевыпуск refresh в БД перед следующим шагом (важно после сброса БД или одиночного запуска папки).
POST /auth/refresh	Получение нового access-токена; обновление ownerToken в переменных.
GET /history (tenant)	Сводная история арендатора: проверка полей messages и deals.

Повторный login перед refresh принципиально важен: если БД была сброшена или папка запускается отдельно, предыдущий refresh-токен может отсутствовать в таблице RefreshToken, что приводит к 401.

7. Папка «Остальной API»

Расширенное покрытие эндпоинтов, не вошедших в сквозной сценарий. Папку необходимо запускать после «Д33 - один проход», так как она использует сохранённые переменные (propertyId, ownerToken и др.).

Запрос / группа	Что проверяется
POST /auth/login (owner + tenant)	Повторный вход; обновление токенов перед расширенными проверками.
POST /auth/password/request-reset	Запрос токена сброса по email арендатора.
POST /auth/password/reset (неверный токен)	Ожидается 401 - негативный сценарий.
GET /me (tenant)	Профиль арендатора.
POST /properties/:id/photos	Добавление фото к объекту; сохранить photoId.
POST /properties/:id/conditions	Добавление условия аренды; сохранить conditionId.
PUT /properties/:id	Обновление объявления (title, price).
POST /deals	Создание заявки на аренду (даты из Pre-request Script).
GET /deals/:id	Получение сделки; проверка статуса PENDING.
PATCH /deals/:id (ACTIVE → COMPLETED)	Смена статуса владельцем.

GET /me/renting	Список сделок арендатора.
GET /me/owning	Объекты и активные сделки владельца.
GET /me/owning/deals	Все сделки по объектам владельца.
GET /history (owner)	Сводная история владельца.
GET /messages (owner, по объекту)	Листинг сообщений со стороны владельца.
DELETE /conditions/:id	Удаление условия аренды.
DELETE /photos/:id	Удаление фото.
POST /auth/logout (owner)	Выход; удаление refresh-сессии из БД.
GET /me (без заголовка Authorization)	Ожидается 401 - негативный сценарий.

8. Наблюдения при тестировании

В процессе прогона коллекции были выявлены и устранены следующие проблемы:

Код	Причина	Решение
401	Refresh-токен не найден в БД (после logout или сброса БД)	Добавлен повторный login перед запросом refresh в папке «Owner»
409	Конфликт при повторных прогонах: активная или PENDING-сделка уже существует на объекте	Доработана бизнес-логика API; уникальные email через timestamp решают конфликт регистрации
500	Зарезервированный алиас type в SQL-запросе PostgreSQL при листинге объектов	Алиас заменён на нейтральный; поправлен QueryBuilder
500	Неоднозначный ORDER BY при JOIN в запросе /history	Добавлены явные алиасы полей в ORDER BY

Вывод

В ходе домашней работы подготовлена Postman-коллекция Rent_Lab1_E2E с тремя папками: сквозным сценарием из 14 шагов, блоком Ownera и расширенным покрытием остальных эндпоинтов. Сквозной сценарий охватывает полный цикл: регистрацию пользователей двух ролей (OWNER, TENANT), создание объявления, публичный листинг и фильтрацию, многоходовую переписку по объекту и прочтение сообщений. Все 25 эндпоинтов Lab1 API покрыты в совокупности трёх папок, включая негативные кейсы (401 без токена, 401 на неверный reset-токен). Использование переменных коллекции и автопроверок во вкладке Tests позволяет многократно прогонять сценарий через Collection Runner без ручного редактирования запросов. В процессе тестирования выявлены и исправлены ошибки на стороне API (конфликты алиасов в SQL, неоднозначный ORDER BY).